Selective repeat

sender

data from above:

- if available seq # in window, send pkt

timeout(n):

- resend pkt n, restart timer

ACK(n) in [sendbase,sendbase+N]:

- mark pkt n as received
- if n oldest unACKed pkt, advance window base to next unACKed seq #
- restart timer for next unACKed seq #

receiver

pkt n in [rcvbase, rcvbase+N-1]

- send ACK(n)
- out-of-order: buffer
- in-order: delivery (like GBN, deliver buffered, in-order pkts), advance window to next not-yet-received pkt

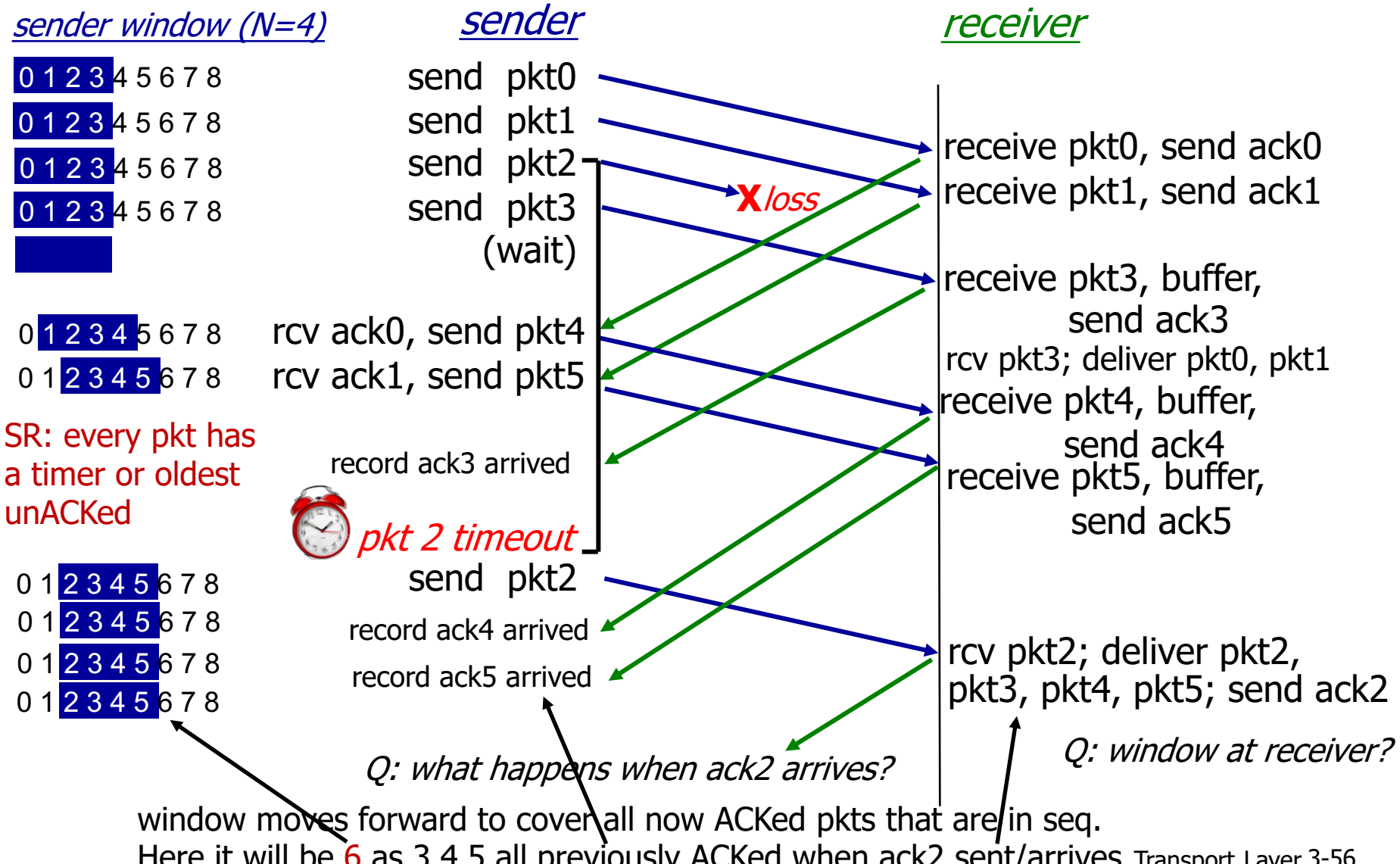
pkt n in [rcvbase-N,rcvbase-1]

- ACK(n) (ACK lost, repeat)

otherwise:

- ignore

Selective repeat in action



Selective repeat: dilemma

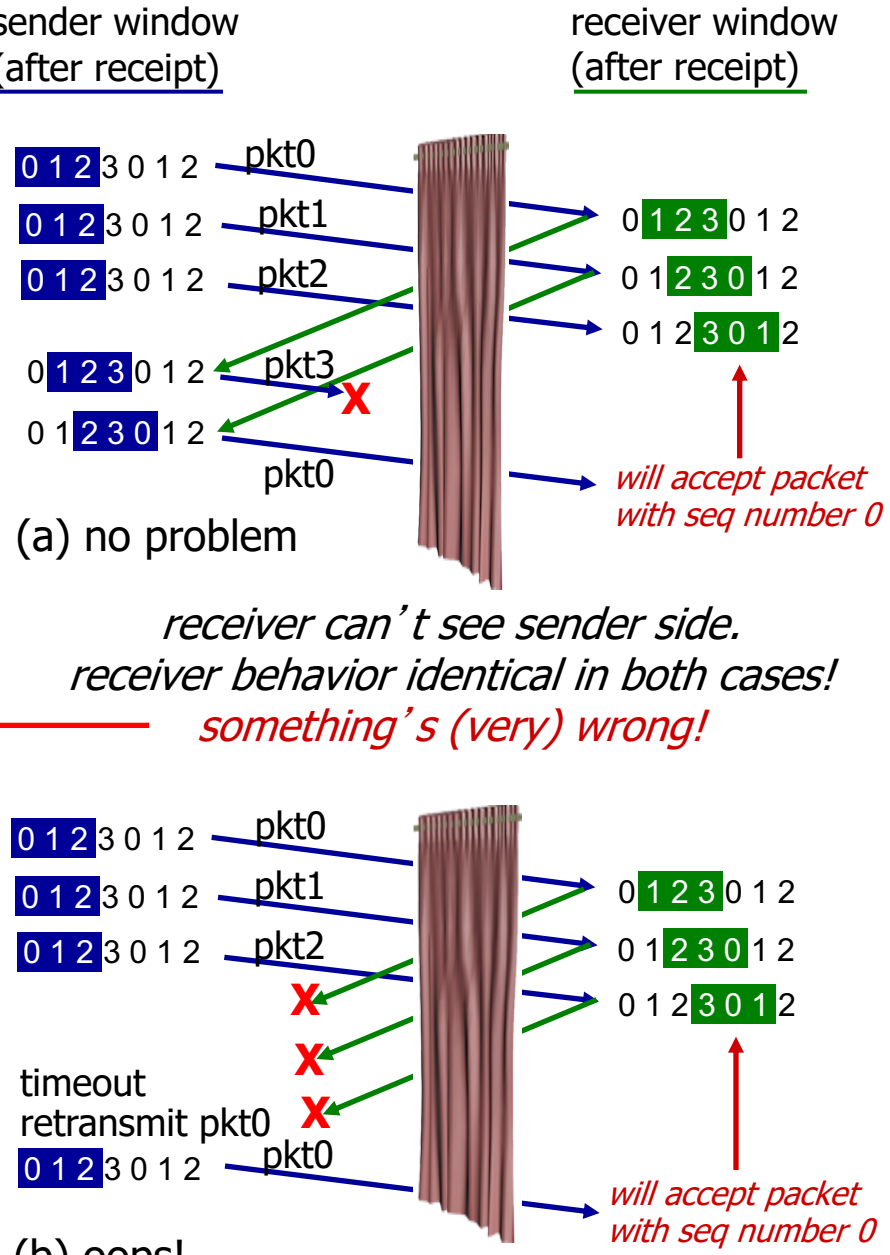
example:

- seq #'s: 0, 1, 2, 3
- window size=3

- receiver sees no difference in two scenarios!
- duplicate data accepted as new in (b)

Q: what relationship between seq # size and window size to avoid problem in (b)?

A: window size must be << than seq # space → half or less



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

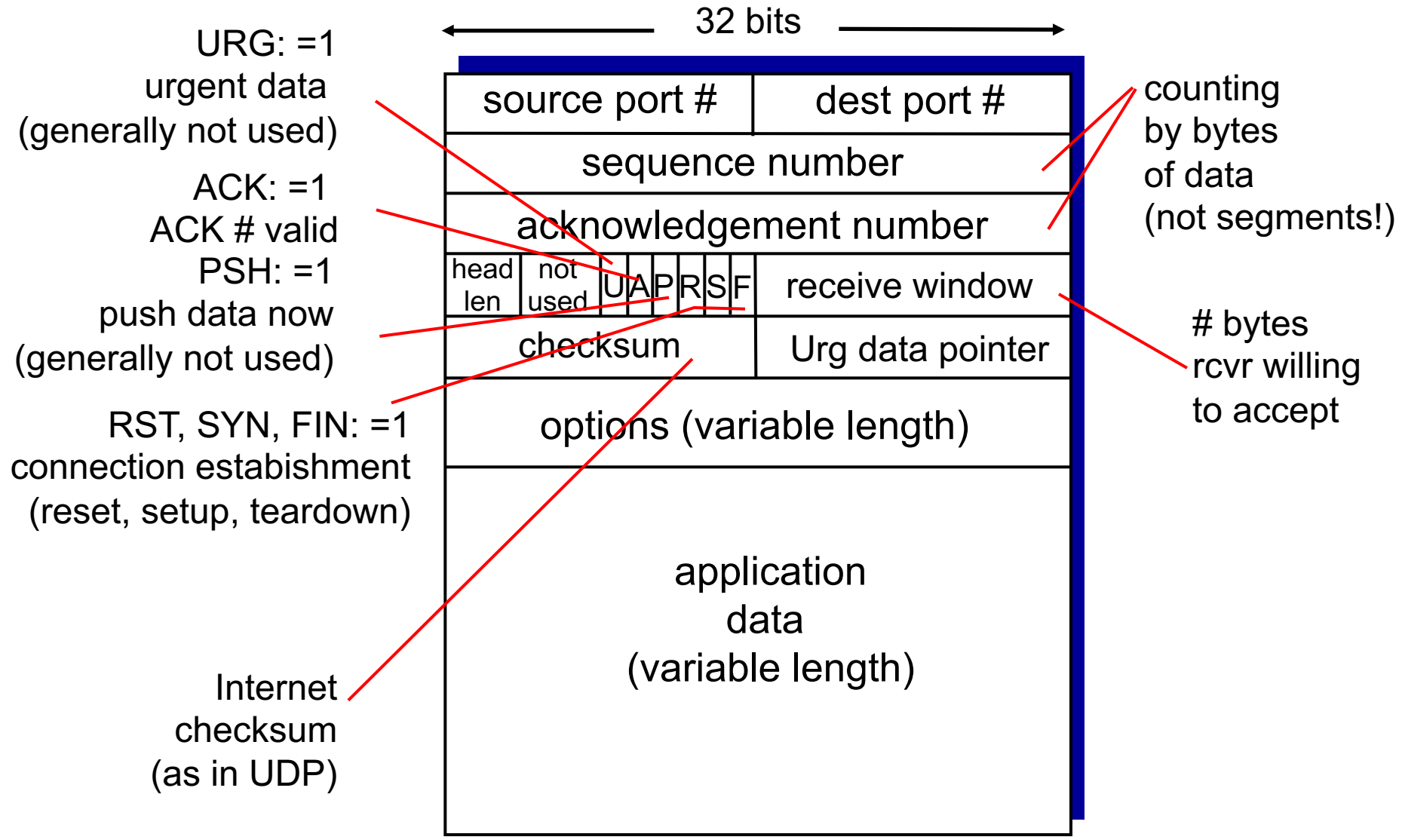
3.6 principles of congestion control

3.7 TCP congestion control

TCP: Overview RFCs: 793,1122,1323, 2018, 2581

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte stream:*
  - no "message boundaries"
- pipelined:
  - TCP congestion and flow control set window size
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- connection-oriented:
  - handshaking (exchange of control msgs) inits sender, receiver state before data exchange
- flow controlled:
  - sender will not overwhelm receiver

TCP segment structure

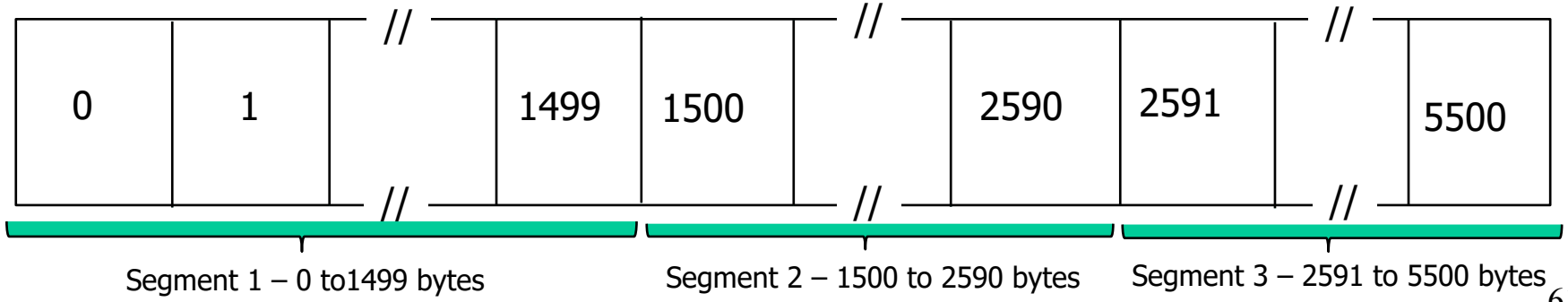


Sequence Nos

- sequence number is 32 bits long
  - the range of SeqNo is

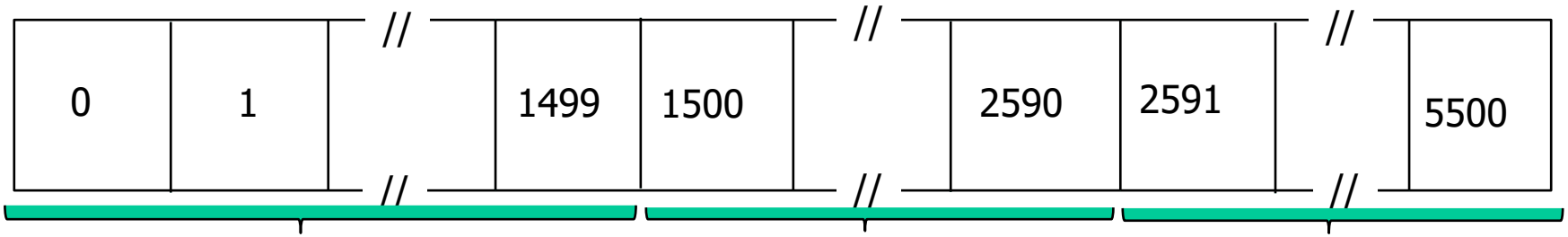
$$0 <= \text{SeqNo} <= 2^{32} - 1 \approx 4.3 \text{ Gbyte}$$

  - a sequence number identifies a specific byte in the byte stream. Each byte has a sequence number
  - an Initial Sequence Number (ISN) for a new connection is picked randomly at each end of the connection and is exchanged during connection establishment



Acknowledgement Nos

- ACKs can be piggybacked
  - a segment from A -> B can contain an acknowledgement for data sent in the B -> A direction
- a hosts uses the acknowledgment field to ACK a pkt
  - if a host sends an ACK# in a segment it sets the "**ACK flag**"
- the ACK# contains the **next** Seq# that a receiving host is expecting to receive from the sender.
  - the ACK# for a segment with Seq# 0 and data length of 1500 bytes (0-1499) is = 1500
  - next byte TCP receiver is expecting is byte # 1500 (received 0-1499)
  - next segment sent from sender should have Seq# = 1500.



TCP seq. numbers, ACKs

sequence numbers:

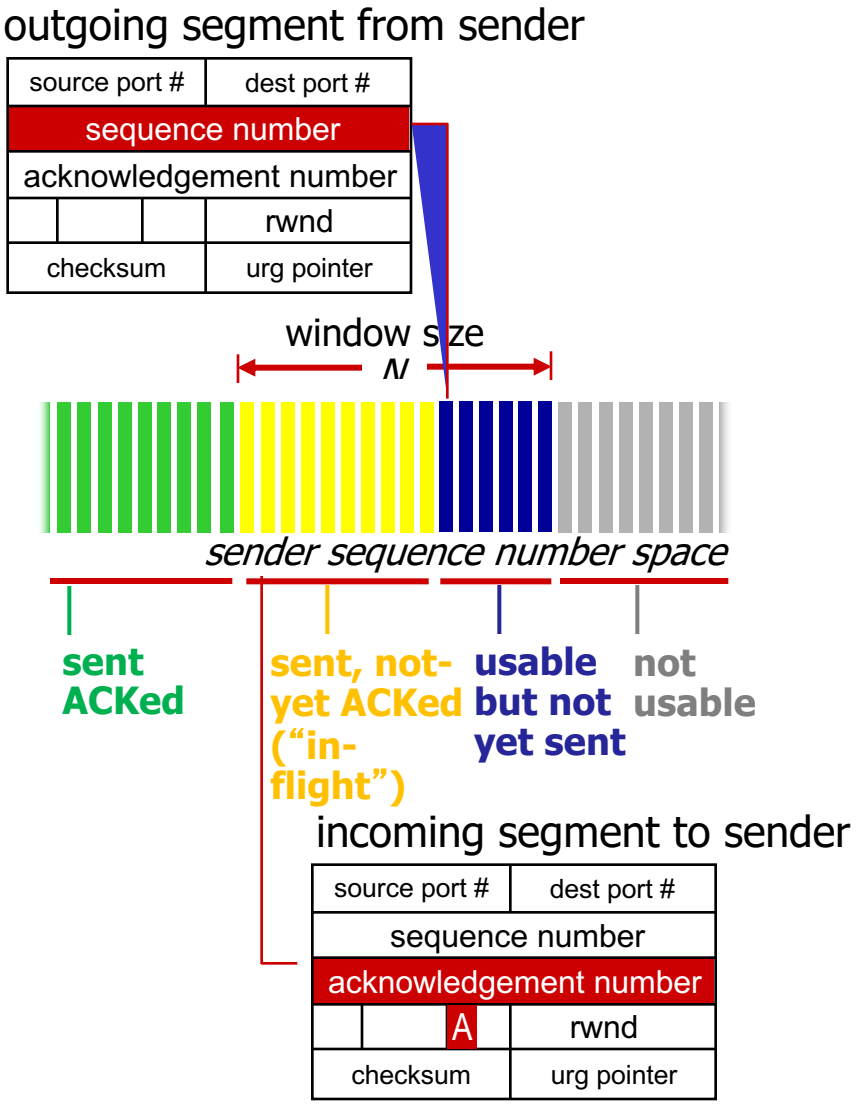
- byte stream "number" of first byte in segment's data

acknowledgements:

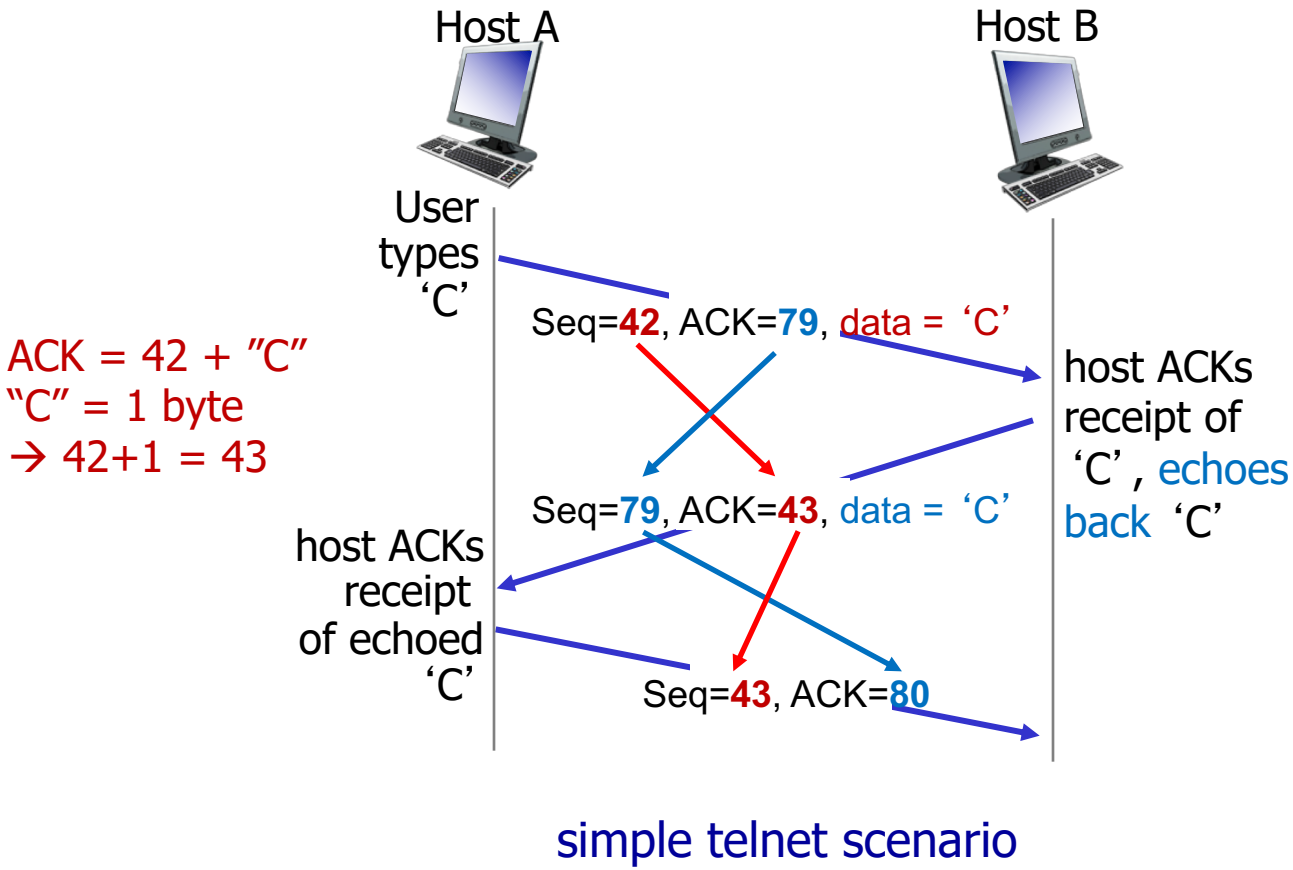
- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments?

A: TCP spec doesn't say, - up to implementor



TCP seq. numbers, ACKs



simple telnet scenario

TCP round trip time, timeout

Q: how to set TCP timeout value?

- longer than RTT
  - but RTT varies
- *too short*: premature timeout, unnecessary retransmissions
- *too long*: slow reaction to segment loss

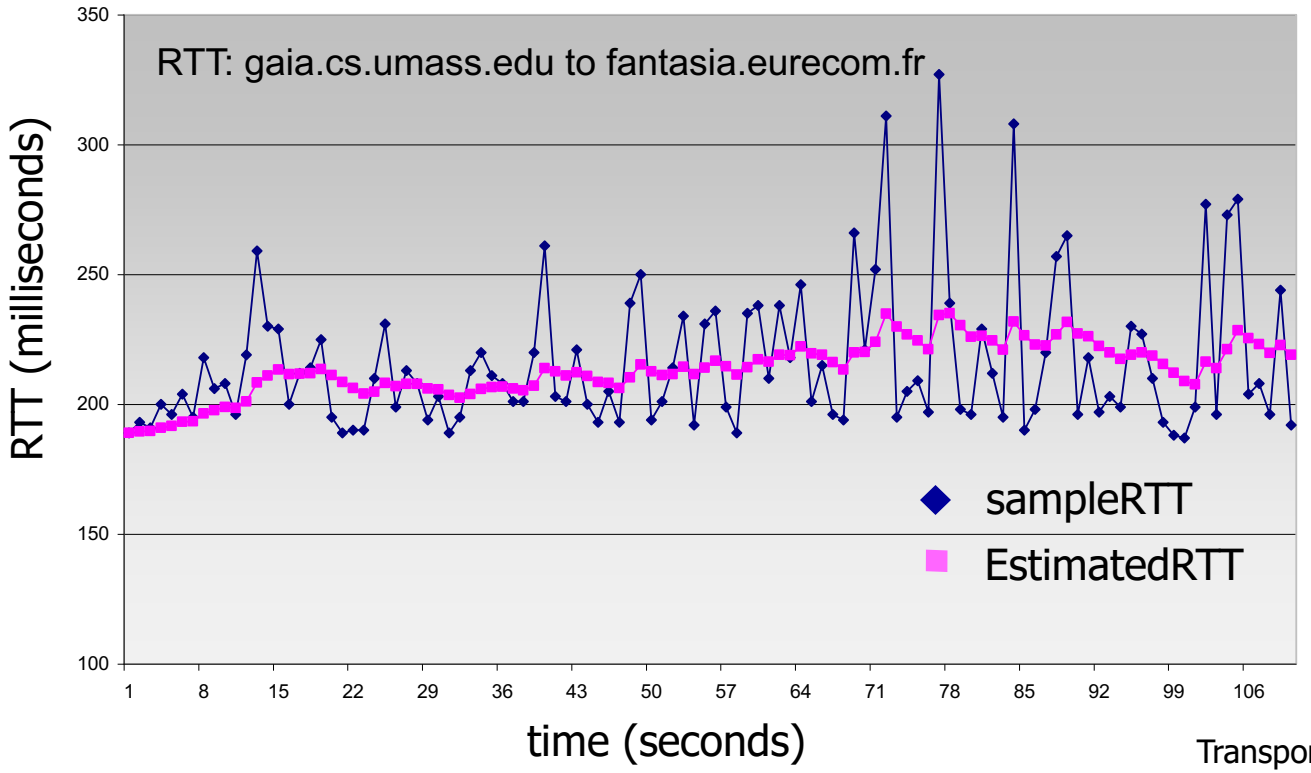
Q: how to estimate RTT?

- **`SampleRTT`**: measured time from segment transmission until ACK receipt
  - ignore retransmissions
- **`SampleRTT`** will vary, want estimated RTT "smoother"
  - average over several *recent* measurements, not just current **`SampleRTT`**

TCP round trip time, timeout

EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT

- exponential weighted moving average
- influence of past sample decreases exponentially fast
- typical value: α = 0.125



TCP round trip time, timeout

- timeout interval: **EstimatedRTT** plus "safety margin"
  - large variation in **EstimatedRTT** -> larger safety margin
- estimate SampleRTT deviation from EstimatedRTT:

DevRTT = (1-β)*DevRTT +
β*|SampleRTT-EstimatedRTT|

(typically, β = 0.25)

TimeoutInterval = EstimatedRTT + 4*DevRTT

estimated RTT

"safety margin"

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

TCP reliable data transfer

- TCP creates a reliable service on top of IP's unreliable service
  - pipelined segments
  - cumulative acks
  - single retransmission timer
- retransmissions triggered by:
  - timeout events
  - duplicate acks

let's initially consider simplified TCP sender:

- ignore duplicate acks
- ignore flow control, congestion control

TCP sender events:

data rcvd from app:

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
  - think of timer as for oldest unacked segment
  - expiration interval: `TimeOutInterval`

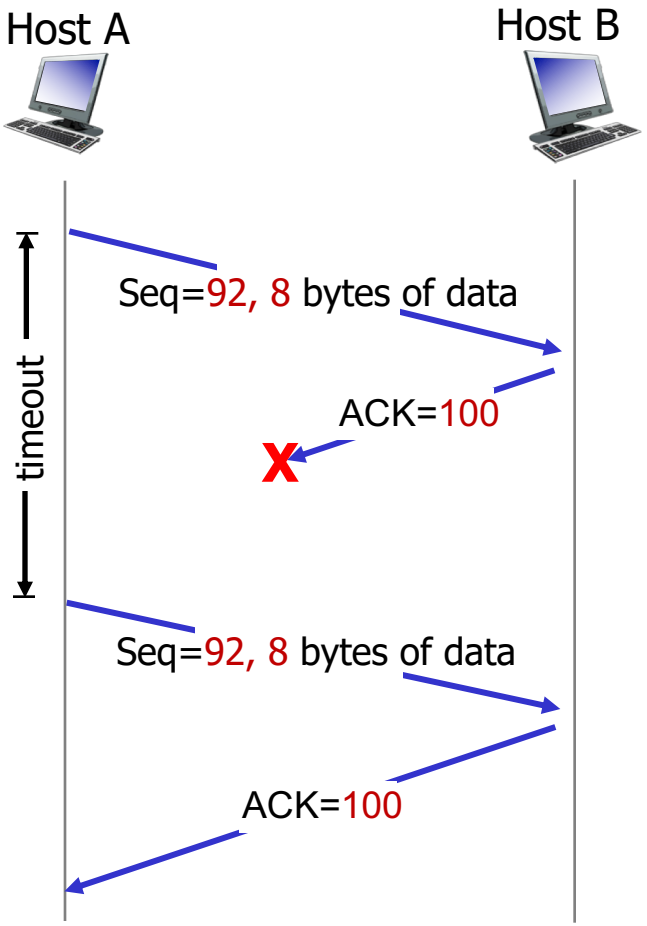
timeout:

- retransmit segment that caused timeout
- restart timer

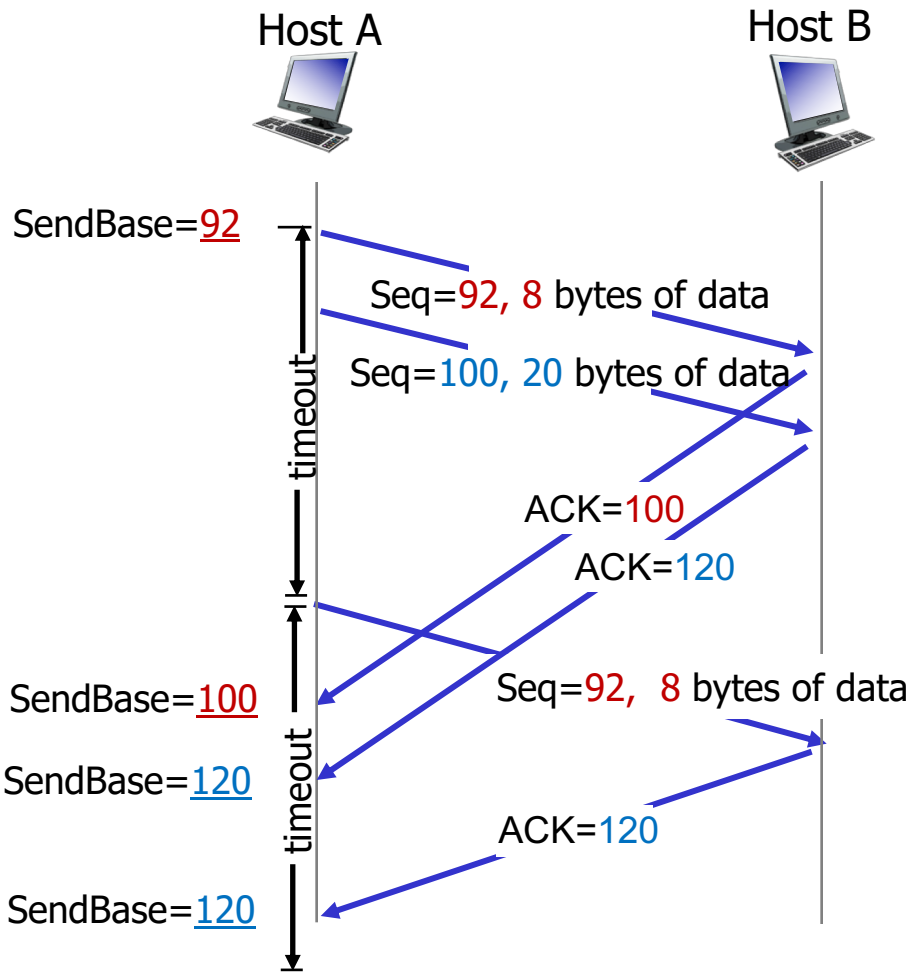
ack rcvd:

- if ack acknowledges previously unacked segments
  - update what is known to be ACKed
  - start timer if there are still unacked segments

TCP: retransmission scenarios

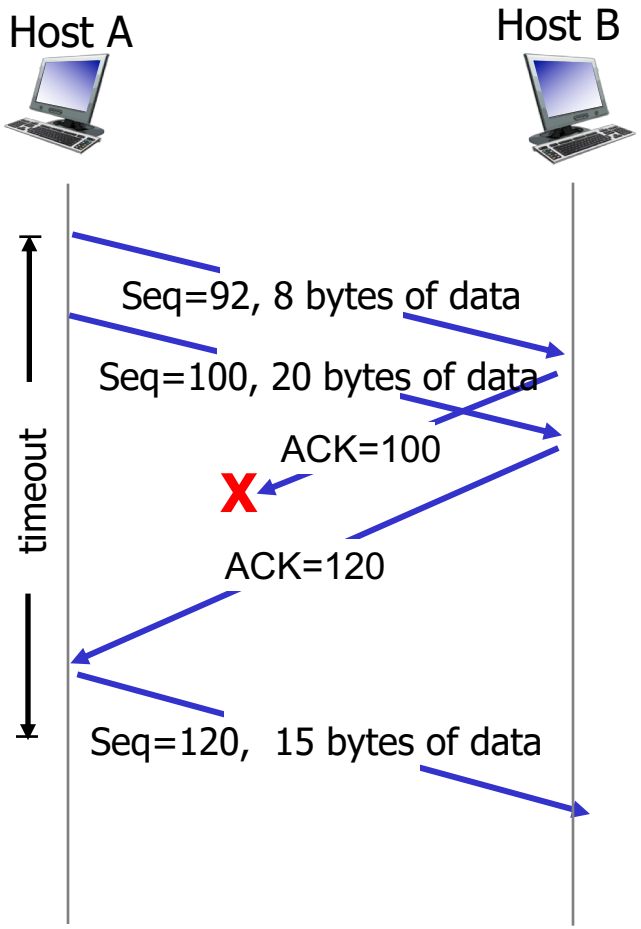


lost ACK scenario



premature timeout

TCP: retransmission scenarios



timeout better estimate, cumulative ACK

TCP ACK generation [RFC 1122, RFC 2581]

event at receiver	*TCP receiver action*
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send *duplicate ACK*, indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

TCP fast retransmit

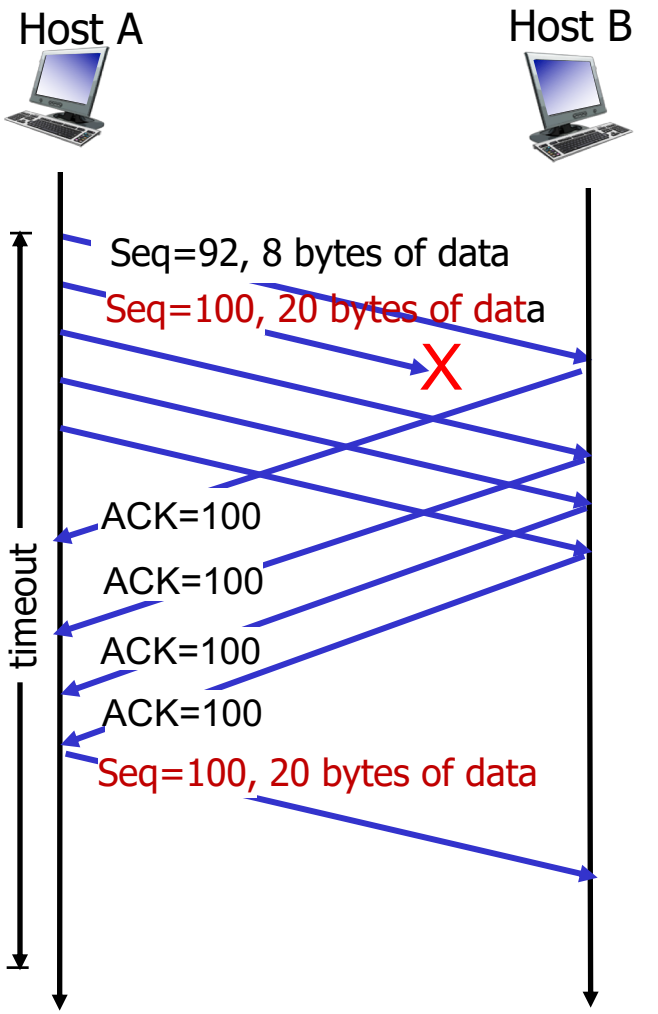
- time-out period often relatively long:
  - long delay before resending lost packet
- detect lost segments via duplicate ACKs.
  - sender often sends many segments back-to-back
  - if segment is lost, there will likely be many duplicate ACKs (every out of order segment will generate an ACK).

TCP fast retransmit

- if sender receives 1+3 ACKs for same data (i.e., original + "triple or 3" duplicate ACKs),
- resend unacked segment with smallest seq #
  - likely that unacked segment lost, so don't wait for timeout

Note: "triple" duplicate ACKs, means one ACK and 2 duplicates of that ACK

TCP fast retransmit



fast retransmit after sender receipt of triple duplicate ACK

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

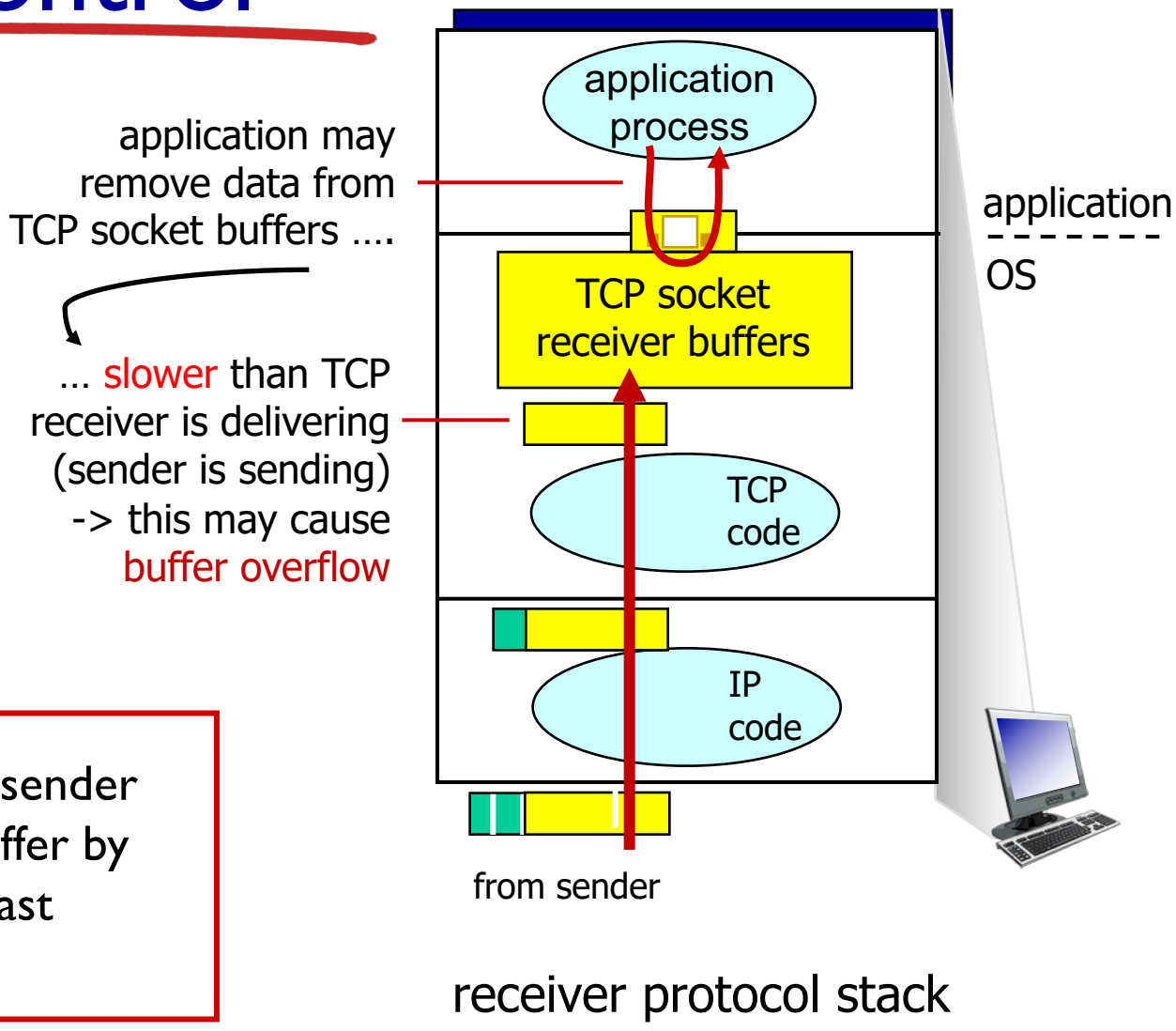
3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

TCP flow control



receiver protocol stack

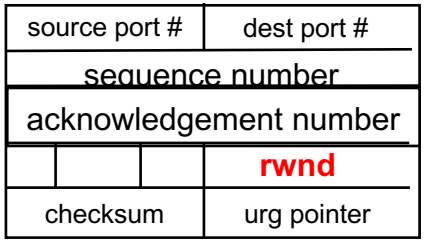
flow control

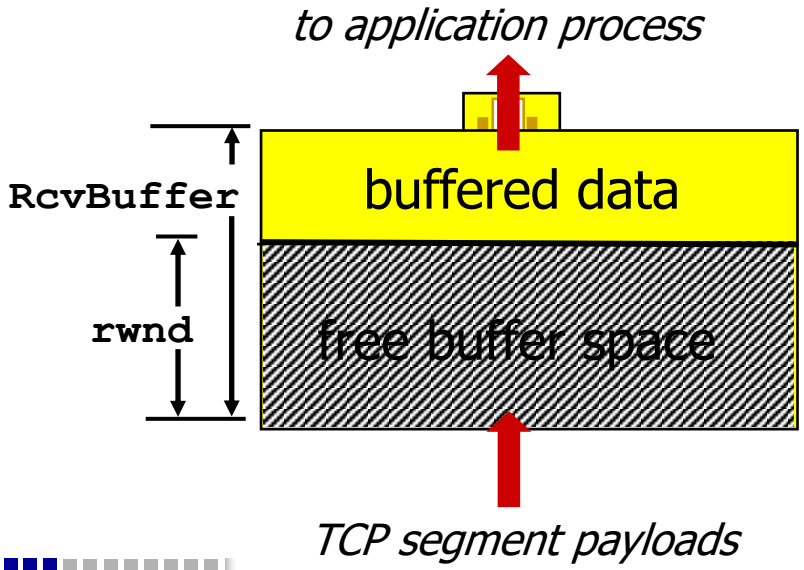
receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast

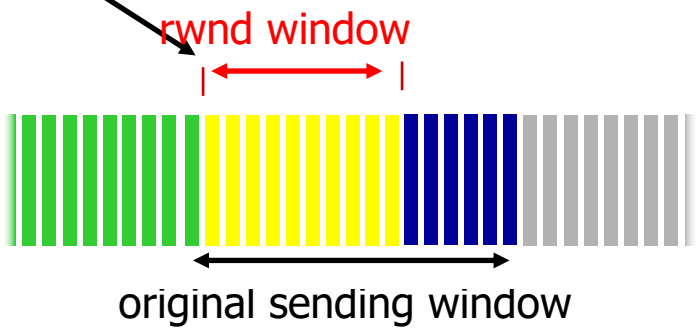
TCP flow control (FC)

- receiver "advertises" free buffer space by including **rwnd** value in TCP header of receiver-to-sender segments
  - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
  - many operating systems auto adjust **RcvBuffer**
- sender limits amount of unacked ("in-flight") data to receiver's **rwnd** value (changes value of sending window → FC window = rwnd)
- guarantees receive buffer will not overflow

segment sent to sender from receiver





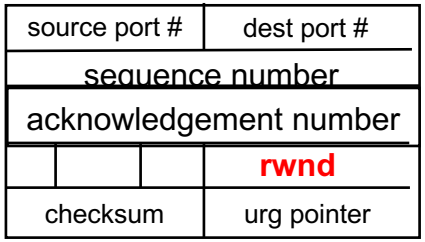


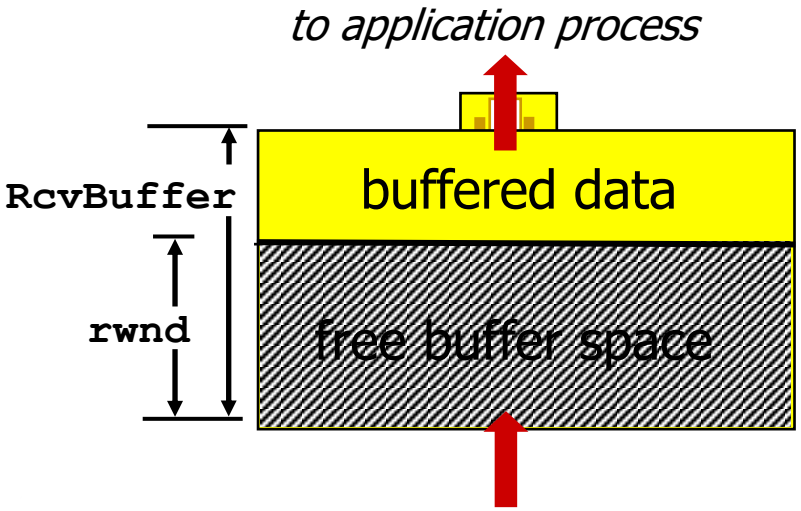
receiver-side buffering

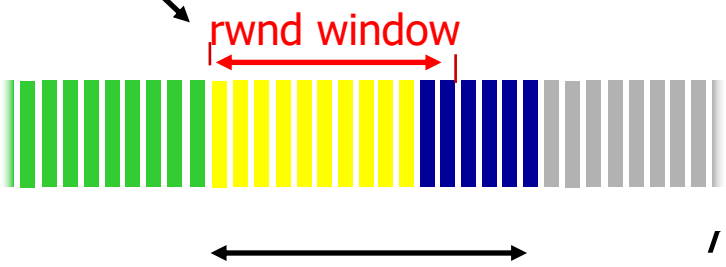
TCP flow control (FC)

- receiver "advertises" free buffer space by including **rwnd** value in TCP header of receiver-to-sender segments
  - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
  - many operating systems auto adjust **RcvBuffer**
- sender limits amount of unacked ("in-flight") data to receiver's **rwnd** value (changes value of sending window → FC window = rwnd)
- guarantees receive buffer will not overflow

segment sent to sender from receiver



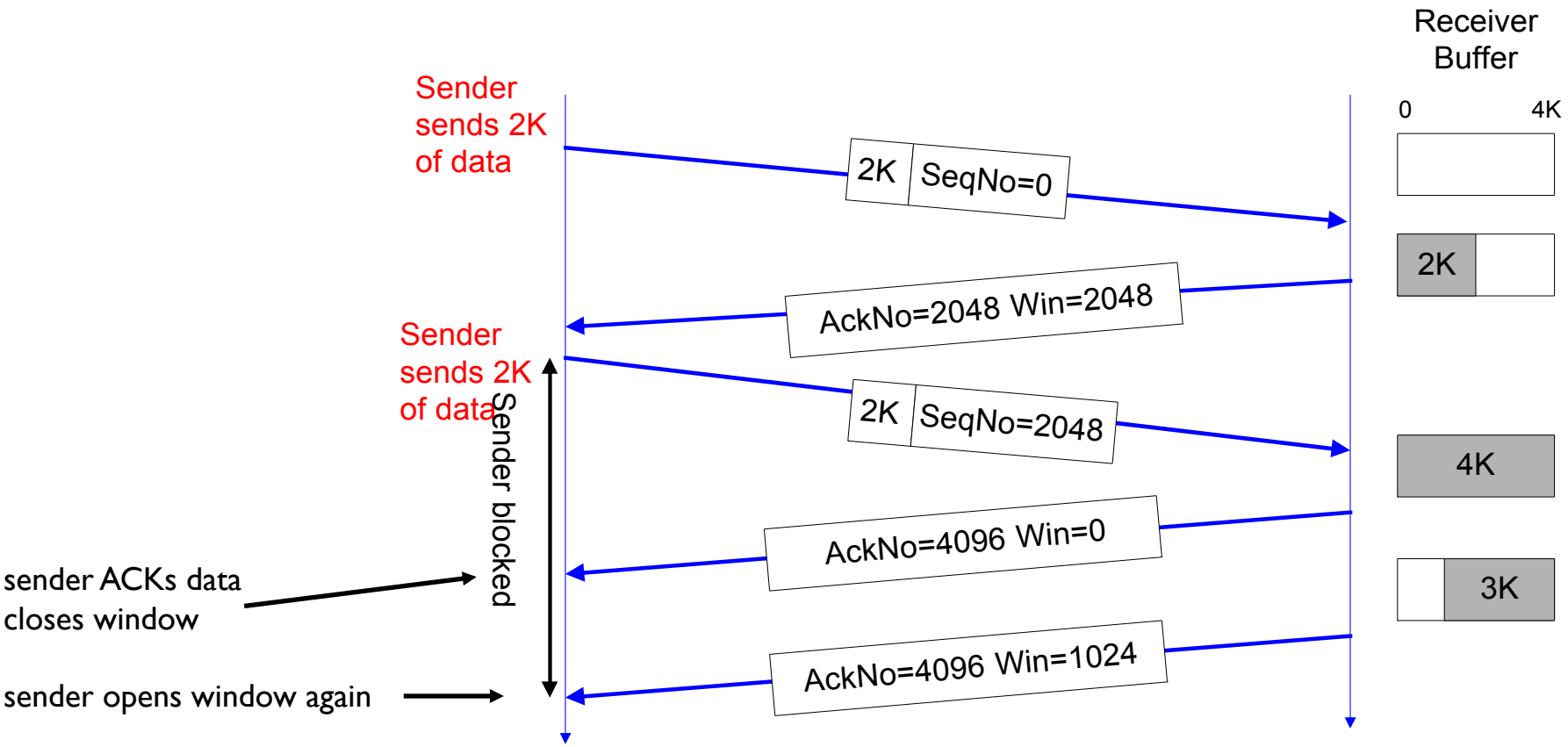




if rwnd > unACKed, then sender can send more packets up to rwnd

receiver-side buffering

Example


Receiver Buffer
0
4K
Sender sends 2K of data
2K
SeqNo=0
2K
AckNo=2048 Win=2048
Sender sends 2K of data
2K
SeqNo=2048
4K
Sender blocked
AckNo=4096 Win=0
3K
sender ACKs data closes window
AckNo=4096 Win=1024
sender opens window again

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

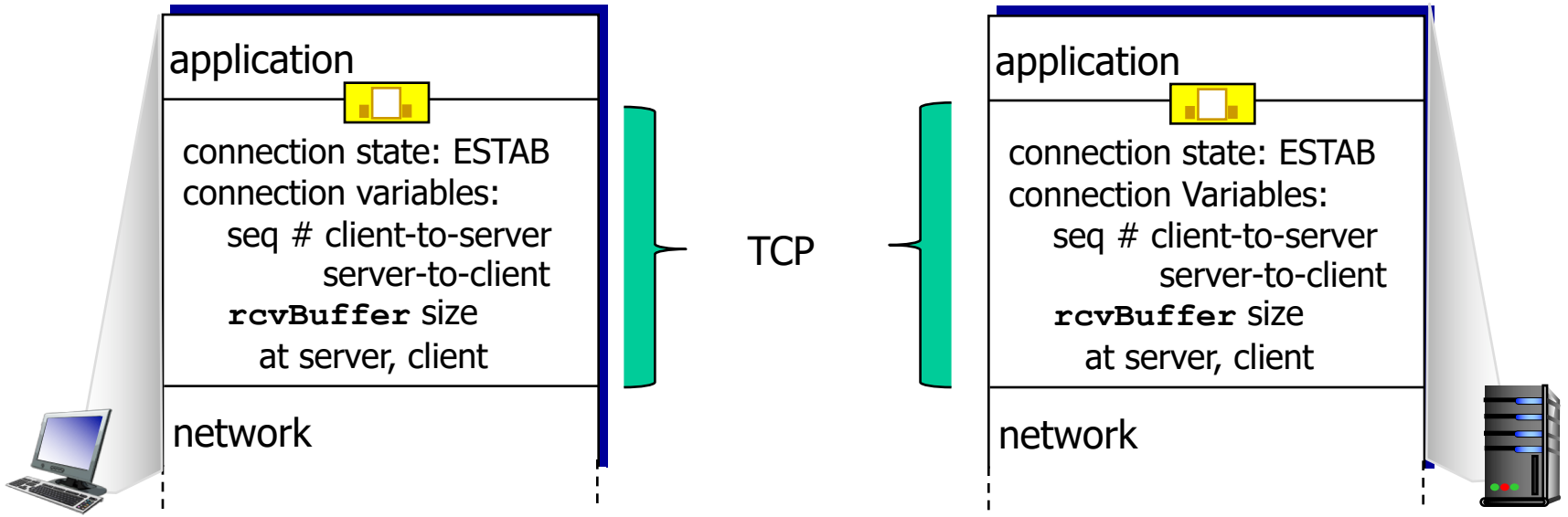
3.6 principles of congestion control

3.7 TCP congestion control

Connection Management

before exchanging data, sender/receiver "handshake":

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters

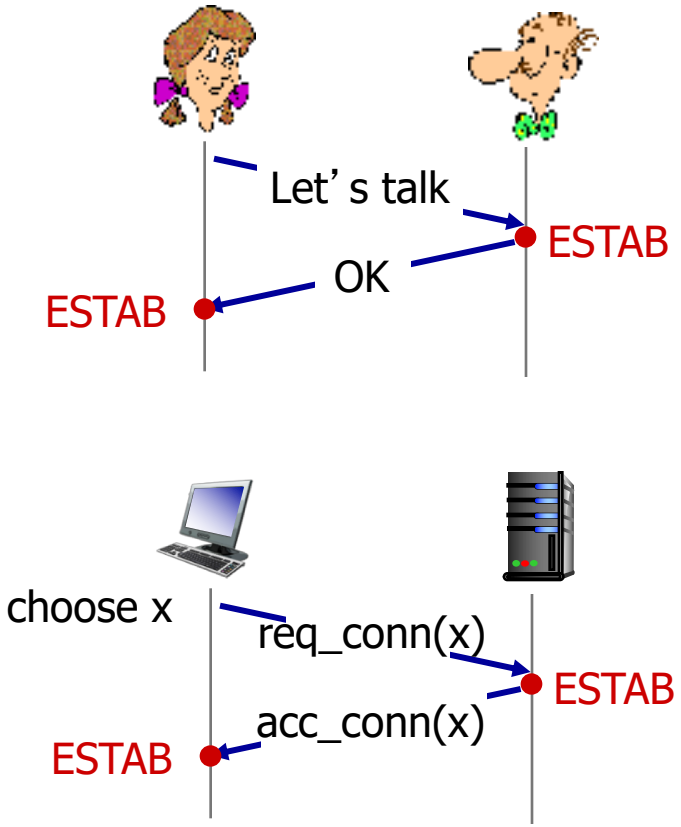



```
Socket clientSocket =
  newSocket("hostname","port
   number");
```

```
Socket connectionSocket =
   welcomeSocket.accept();
```

Agreeing to establish a connection

2-way handshake:

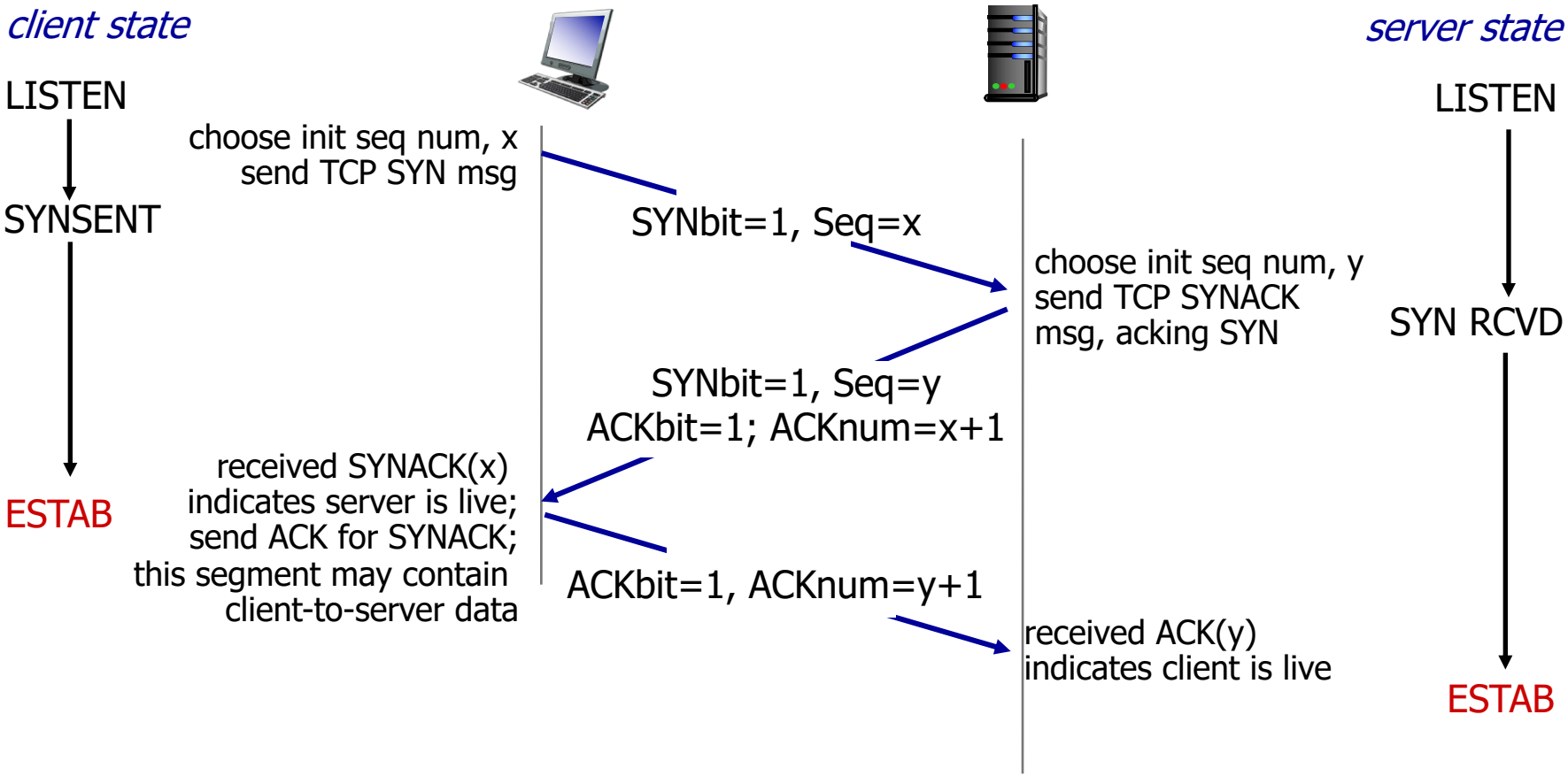


Q: will 2-way handshake always work in network?

A: No

- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

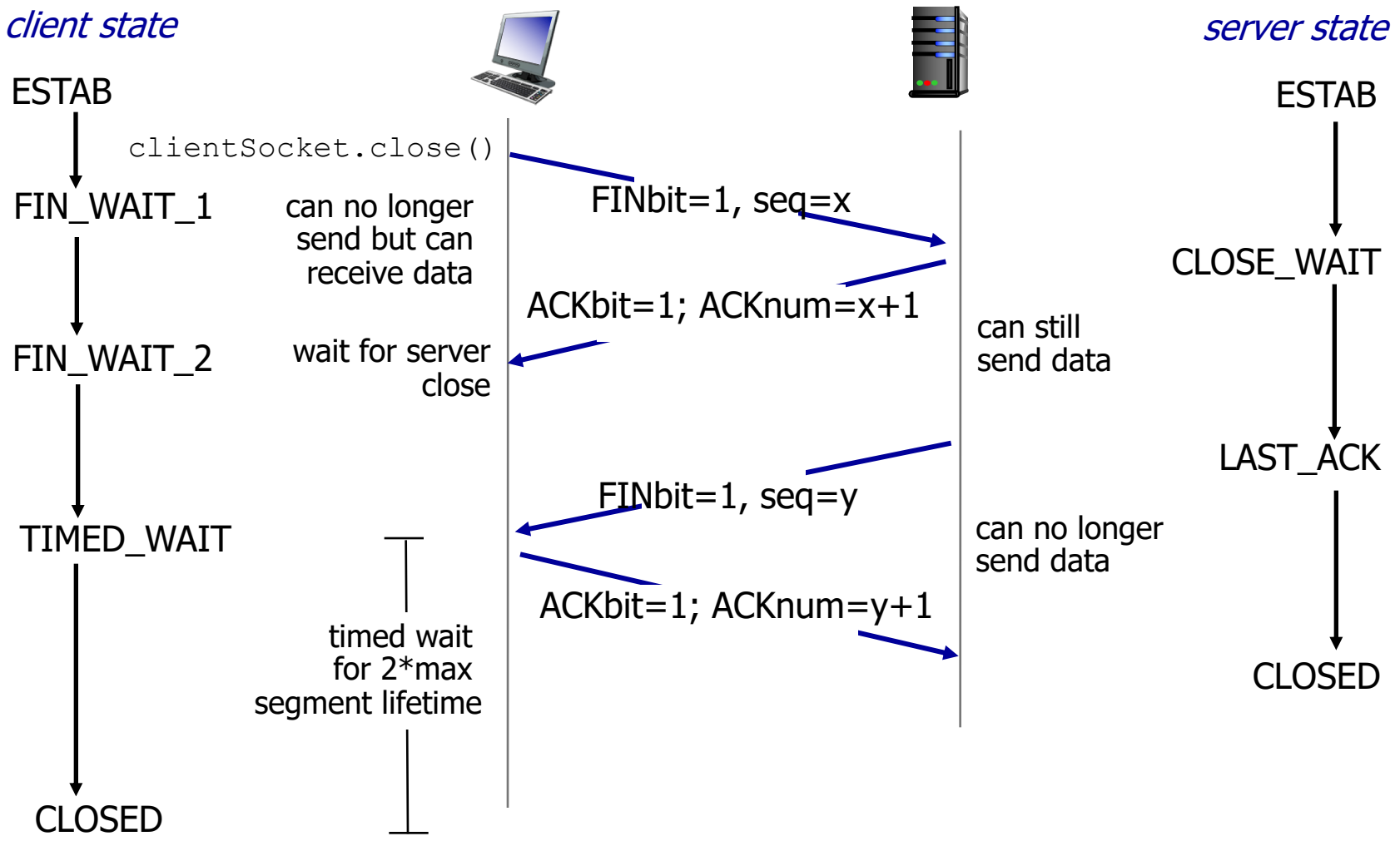
TCP 3-way handshake



TCP: closing a connection

- client, server each close their side of connection
  - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
  - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

TCP: closing a connection



Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

Principles of congestion control

congestion:

- informally: "too many sources sending too much data too fast for *network* to handle"
- different from flow control!
- manifestations:
  - lost packets (buffer overflow at routers)
  - long delays (queueing in router buffers)

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

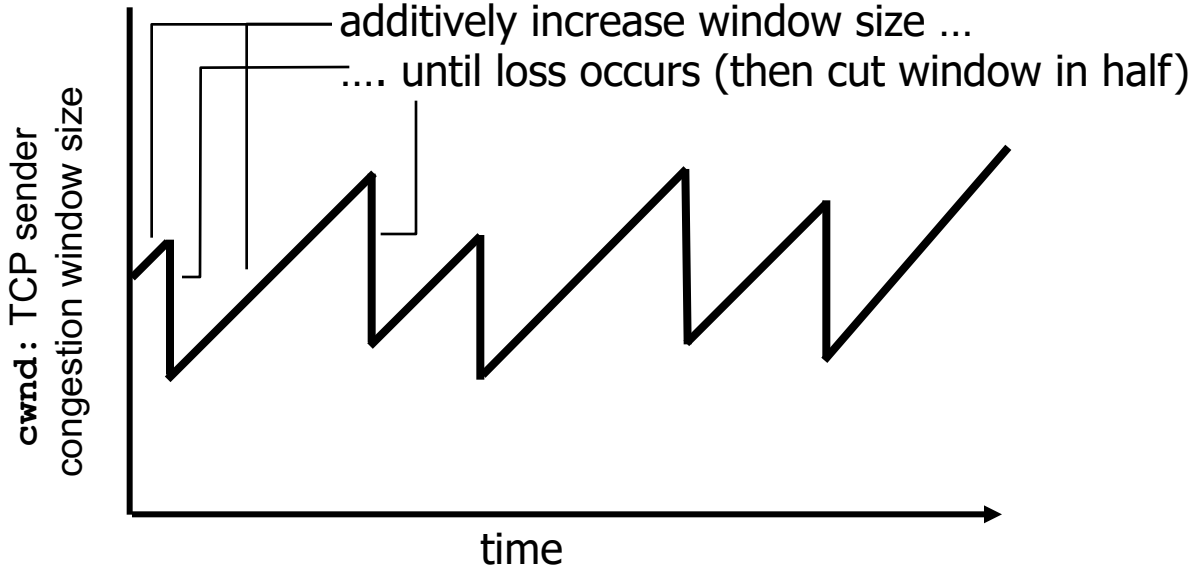
3.6 principles of congestion control

3.7 TCP congestion control

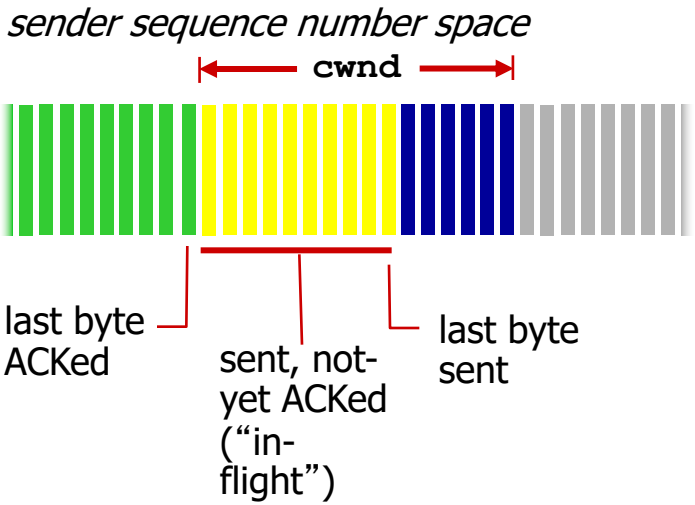
TCP congestion control: principles

- *approach:* sender increases transmission rate (window size), probing for usable bandwidth, until loss occurs
  - *Additive Increase:* increase **`cwnd`** by 1 MSS every RTT (i.e., received ACK) until loss detected
  - *Multiplicative Decrease:* cut **`cwnd`** in half after loss

AIMD saw tooth behavior: probing for bandwidth



TCP Congestion Control: details



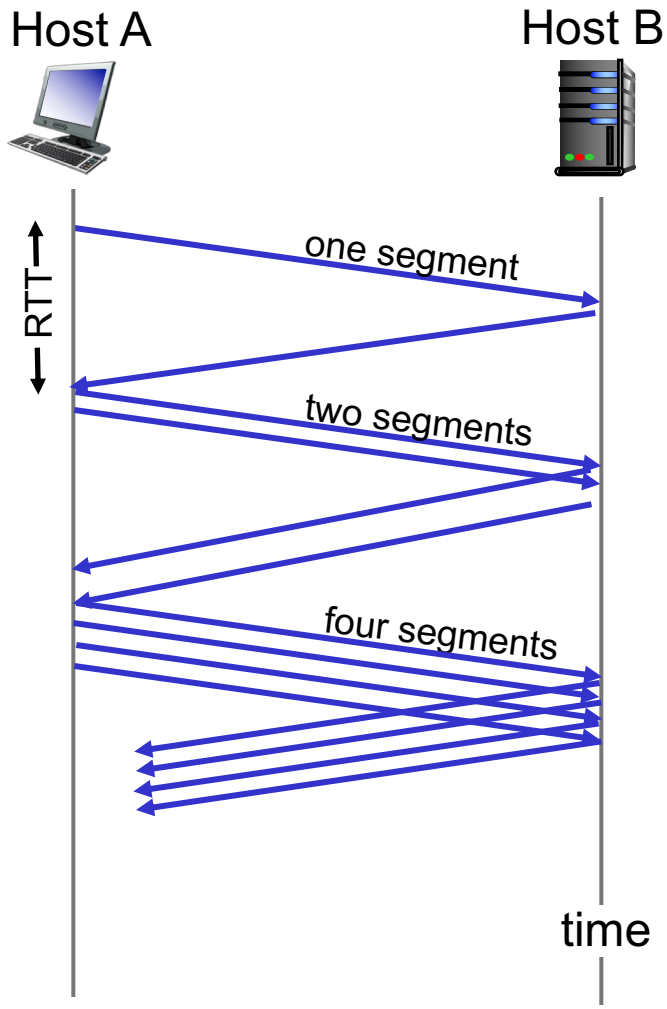
- sender limits transmission:

$$\texttt{LastByteSent} - \texttt{LastByteAcked} \leq \texttt{cwnd}$$

- **cwnd** is dynamic, function of perceived network congestion

TCP Slow Start

- when connection begins, increase rate exponentially until first loss event or threshold is reached, after which **`cwnd`** grows more slowly (linearly - congestion avoidance CA phase):
  - initially **`cwnd`** = 1 MSS
  - double **`cwnd`** every RTT
  - done by incrementing **`cwnd`** for every ACK received
- *summary:* initial rate is slow but ramps up exponentially fast – 1, 2, 4..
- linear growth after threshold



TCP: detecting, reacting to loss

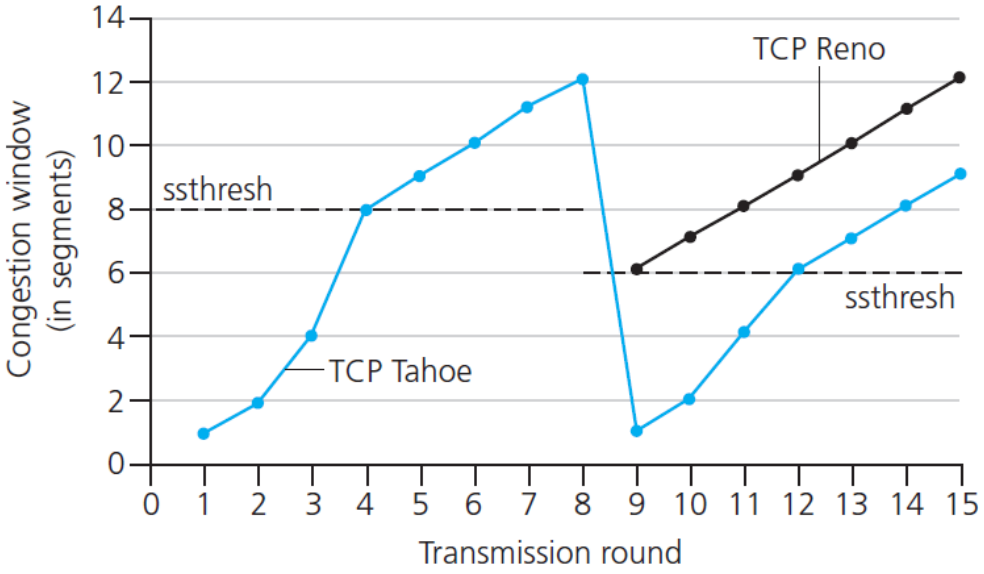
- loss indicated by timeout:
  - **`cwnd`** set to 1 MSS again;
  - window then grows exponentially (as in slow start) to threshold, then grows linearly
- loss indicated by 3 duplicate ACKs (1+3 in total):
  - TCP Tahoe sets **`cwnd`** to 1 MSS and goes to slow start again (same behavior as for timeout)
  - resends packet
- TCP RENO
  - dup ACKs indicate network capable of delivering some segments
  - **`cwnd`** is cut in half window
  - resends packet
  - goes into fast recovery – if it receives ACK it stays in linear mode, else it goes into slow start.

TCP: switching from slow start to CA

Q: when should the exponential increase switch to linear?

A: when **`cwnd`** gets to a threshold.

Note: here we show Reno in fast recovery mode.

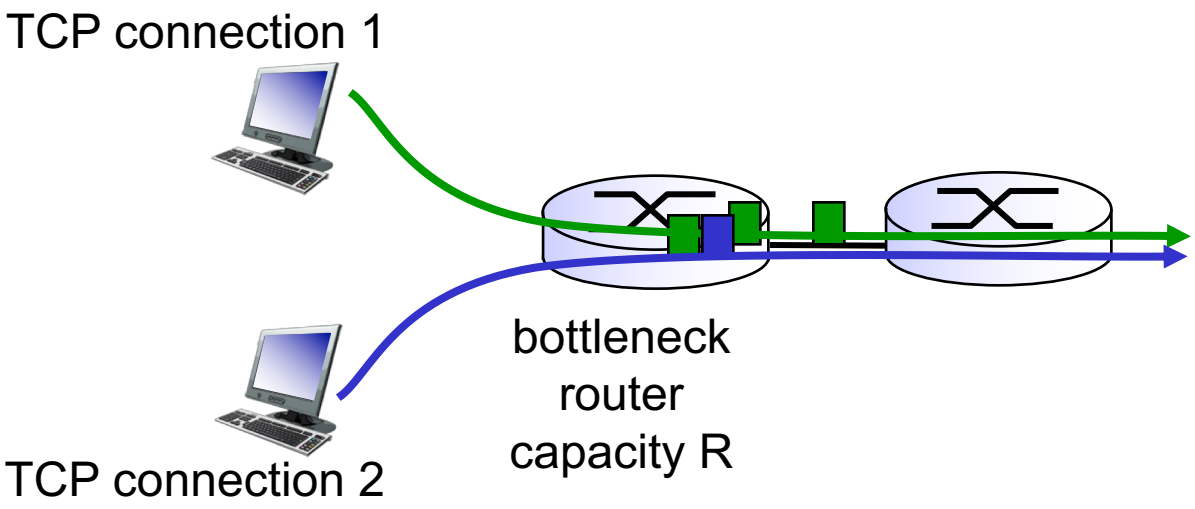


Implementation:

- variable **`ssthresh`**, starts with default value
- on loss event, **`ssthresh`** is set to 1/2 of **`cwnd`** just before loss event

TCP Fairness

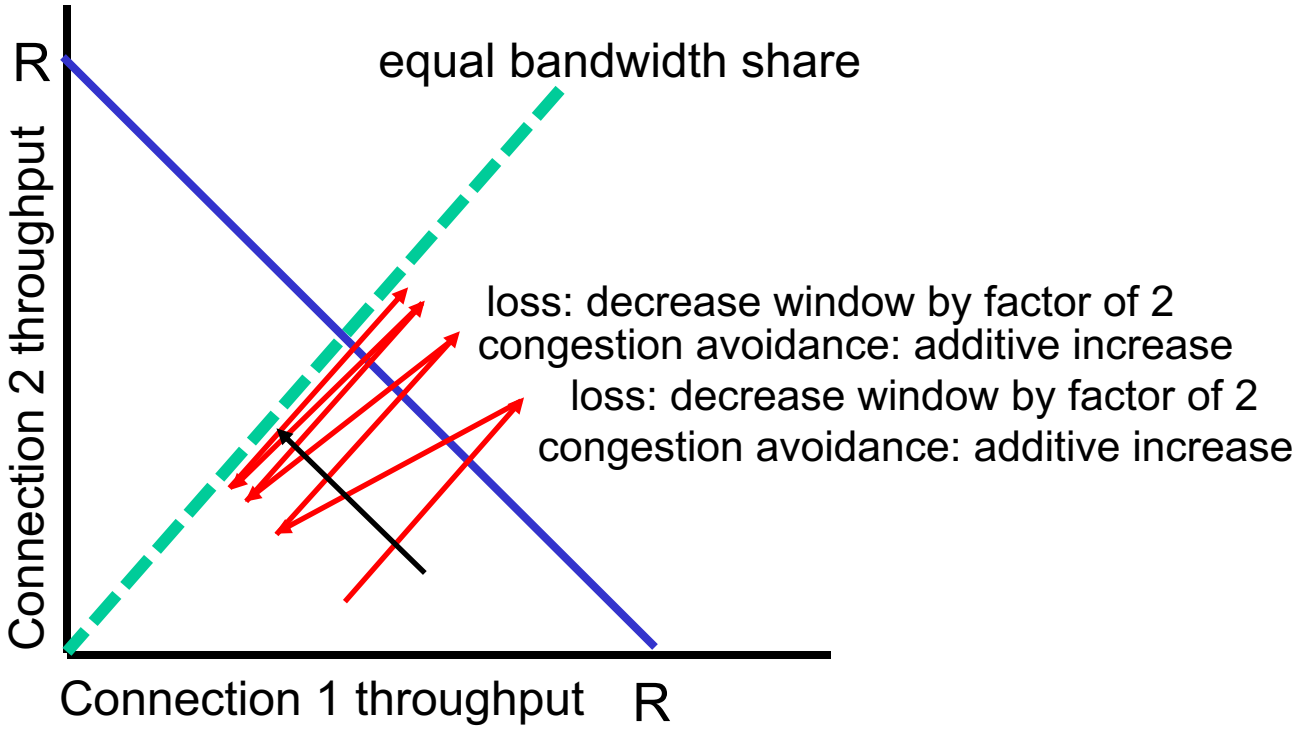
fairness goal: if K TCP sessions share same bottleneck link of bandwidth R, each should have average rate of R/K



Why is TCP fair?

two competing sessions:

- additive increase gives slope of 1, as throughout increases
- multiplicative decrease decreases throughput proportionally



Fairness (more)

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss

Fairness, parallel TCP connections

- application can open multiple parallel connections between two hosts
- web browsers do this
- e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10 (1/(9+1=10)R)
  - new app asks for 11 TCPs, gets ~ R/2 (11/(9+11=20)R)

Explicit Congestion Notification (ECN)

network-assisted congestion control:

- two end points negotiated ECN capability in setup phase with options in SYN, SYN ACK packets)
- two bits in IP header (ToS field) marked *by network router* to indicate congestion capability (01 or 10) and congestion (11)
- congestion indication carried to receiving host (packet not dropped)
- receiver (seeing 11 in IP datagram) sets ECE bit on receiver-to-sender ACK segment to notify sender of congestion (unused flag fields in TCP)
- sender reduces cwnd and sets CRW bit (second unused flag bit) on next segment transmission